UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Amazon Engineering Challenge

ÁRVORES BALANCEADAS: AVL E RUBRO-NEGRA

Rafael Medeiros Câmara da Silva | Matrícula 20250067640

IMD0039 - Estruturas de Dados Básicas II

Instituto Metrópole Digital - UFRN | Trabalho da Unidade II

A proposta

1

Contexto: Black Friday na Amazon. O sistema de logística precisa registrar, localizar e remover identificadores de encomendas com eficiência, em volume muito alto.

Problema: a ordem de chegada dos IDs não é controlada. Em uma árvore binária de busca comum isso pode degenerar a estrutura e comprometer o tempo de resposta.

Objetivos do projeto

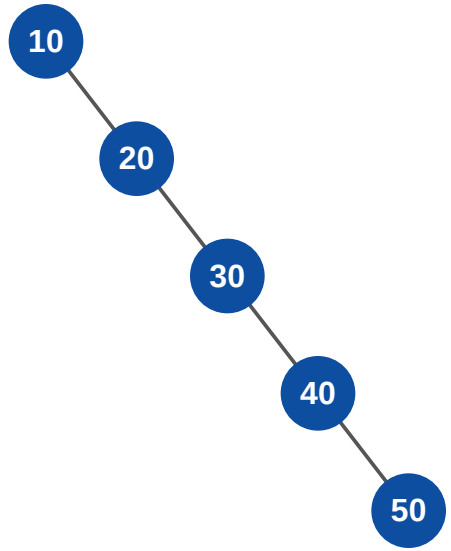
- Implementar do zero uma árvore AVL e uma Rubro-Negra em C++, sem bibliotecas prontas
- Submeter ambas às mesmas cargas e sequências de operações
- Analisar altura final, número de rotações, recolorações e tempo de execução
- Defender uma recomendação técnica para a equipe da Amazon baseada nos testes

O desafio técnico

Uma BST comum não possui mecanismo de balanceamento. Quando pacotes chegam ordenados, a árvore degenera em uma lista encadeada e o custo passa de O(log n) para O(n).

BST sem balanceamento (N = 1200)	Altura obtida	Altura ideal
Inserção crescente	1200	11
Inserção decrescente	1200	11
Inserção aleatória	24	11

BST após inserir 10, 20, 30, 40, 50



O balanceamento mantém a altura em O(log n) para qualquer sequência de entrada. A árvore AVL controla estritamente a diferença de altura; a Rubro-Negra controla o equilíbrio por meio de regras de coloração dos nós.

A solução desenvolvida

- Linguagem: C++17, sem uso de bibliotecas prontas de árvore. Rotações e recolorações feitas do zero
- Menu interativo permite escolher Árvore AVL ou Árvore Rubro-Negra no início da execução
- Operações: inserir, remover, buscar, percurso em ordem, visualizar árvore, métricas e checar balanceamento
- Visualização horizontal no terminal (raiz à esquerda). Na AVL exibe [fb=...]; na Rubro-Negra exibe a cor de cada nó: (V) vermelho e (P) preto
- Contadores internos registram o total de rotações e recolorações executadas

Decisões de implementação

- Chave duplicada: inserção recusada com aviso ao usuário, mantendo a estrutura intacta
- Busca ou remoção de ID inexistente: retorna falso sem alterar a árvore
- Rotação dupla contabilizada como 2 rotações simples nos contadores
- Remoção de nó com 2 filhos: substituição pelo sucessor em ordem (menor da subárvore direit

```
arvore.hpp   (interface base)
avl.hpp      (ArvoreAVL)
rb.hpp       (ArvoreRB)
main.cpp     (menu CLI)
testes.cpp   (suíte de testes)
```

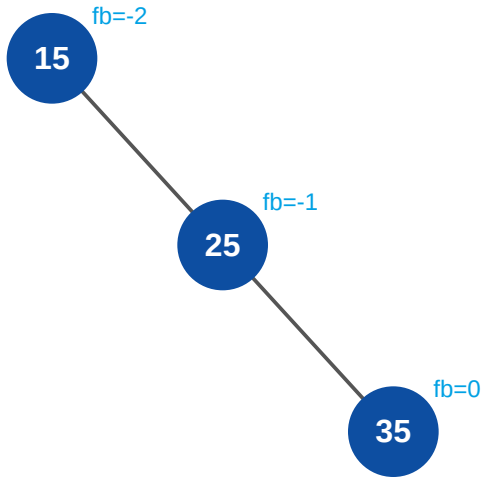
Árvore AVL: estratégia de balanceamento

- Para qualquer nó, o fator de balanceamento fb = h(esq) - h(dir) deve pertencer a {-1, 0, +1}
- Após inserir ou remover, recalcula-se altura e fator no retorno da recursão; se |fb| = 2, rotaciona
- Altura fortemente limitada a ~1,44 log2(n), garantindo árvores mais baixas para busca

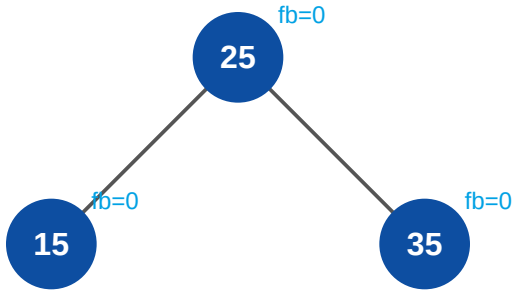
Caso	Condição de disparo	Operação de correção
Esquerda-Esquerda (LL)	fb(nó) = +2 e fb(filho esq) >= 0	Rotação simples à direita
Direita-Direita (RR)	fb(nó) = -2 e fb(filho dir) <= 0	Rotação simples à esquerda
Esquerda-Direita (LR)	fb(nó) = +2 e fb(filho esq) < 0	Rotação dupla (esq no filho + dir no nó)
Direita-Esquerda (RL)	fb(nó) = -2 e fb(filho dir) > 0	Rotação dupla (dir no filho + esq no nó)

AVL: exemplo de rotação (inserir 15, 25, 35)

Antes (após inserir 35): nó 15 com fb = -2, caso Direita-Direita



Depois: rotação simples à esquerda no 15



O nó 25 assume a raiz e o 15 passa a ser seu filho esquerdo. O percurso em ordem (15, 25, 35) é preservado e a altura da árvore reduz de 3 para 2.

Saída real do programa: raiz 25 [fb=0], filhos 15 [fb=0] e 35 [fb=0] | Total de rotações: 1.

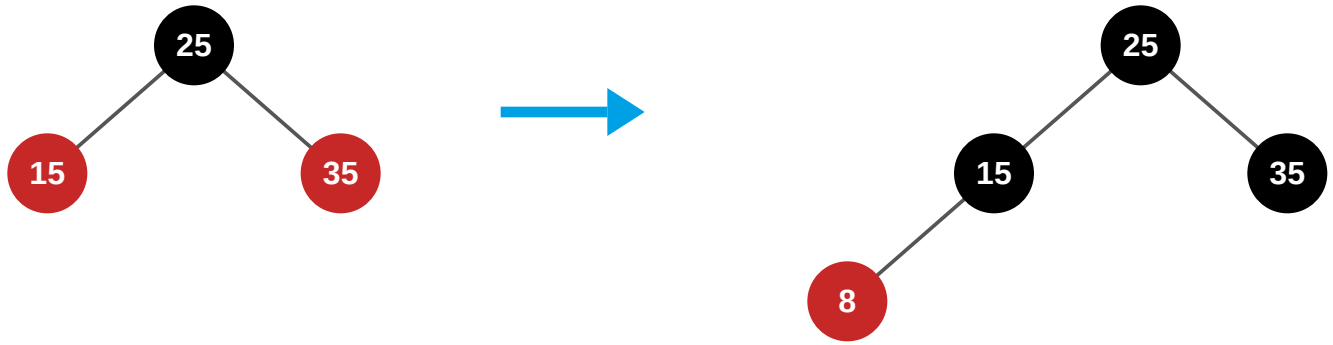
Árvore Rubro-Negra: propriedades

- 1. Todo nó da árvore é colorido como vermelho (V) ou preto (P)
- 2. O nó raiz é obrigatoriamente preto
- 3. Um nó vermelho não pode ter filhos vermelhos (sem conflito vermelho-vermelho)
- 4. Todo caminho de um nó até as folhas nulas (sentinela) contém o mesmo número de nós pretos

- Consequência teórica: o maior caminho nunca excede o dobro do menor, garantindo altura <= 2 log2(n + 1)
- Todo novo nó é inserido como vermelho. Se o pai for vermelho, analisa-se o tio: tio vermelho exige apenas recoloração (pai, tio e avô); tio preto exige rotação (simples ou dupla) seguida de recoloração
- Na exclusão de um nó preto, corrige-se a falta de altura negra com recolorações e no máximo 3 rotações
- Balanceamento mais flexível que o da AVL: reduz rotações de ponteiros em troca de altura levemente maior

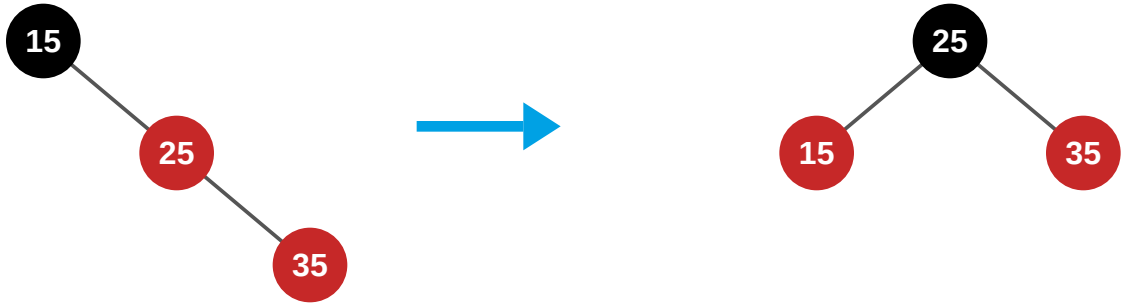
Rubro-Negra: exemplos de ajuste

Recoloração: inserir 8 (pai 15 e tio 35 vermelhos)



1º caso: pai (15) e tio (35) passam a pretos; o avô (25) permaneceria vermelho, mas volta a preto por ser raiz. O nó 8 entra vermelho.

Rotação: inserir 35 após 15 e 25 (tio sentinela = preto, nós em linha)



2º caso: rotação simples à esquerda em 15; o 25 sobe para a raiz (preto) e seus filhos 15 e 35 ficam vermelhos.

Resultado contabilizado pelo programa para a sequência 15, 25, 35, 8: Rotações = 1 | Recolorações = 7.

Demonstração: AVL


```
Modo selecionado: AVL
Pacote 15 registrado.
Pacote 25 registrado.
Pacote 35 registrado.
     35 [fb=0]
25 [fb=0]
     15 [fb=0]
ID 25 localizado no sistema.
ID 99 nao encontrado.
Aviso: ID 99 inexistente.
Aviso: ID 25 duplicado (recusado).
Pacote 25 removido.
15 35
35 [fb=1]
     15 [fb=0]
Total de pacotes: 2
Altura da arvore: 2
Total de rotacoes: 1
Total de recoloracoes: 0
Invariantes preservadas (OK)
```

- Inserção de 15, 25 e 35 aciona rotação simples à esquerda e equilibra a árvore.
- Busca de pacote registrado e inexistente.
- Tentativa de remover chave ausente (99) avisa o usuário sem alterar ponteiros.
- Chave repetida (25) é rejeitada.
- Remoção da raiz (25, dois filhos) promove o sucessor (35) e valida o balanceamento.

Demonstração: Rubro-Negra


```
Modo selecionado: Rubro-Negra
Pacote 15 registrado.
Pacote 25 registrado.
Pacote 35 registrado.
     35(V)
25(P)
     15(V)
Pacote 8 registrado.
     35(P)
25(P)
     15(P)
          8(V)
ID 25 localizado no sistema.
ID 99 nao encontrado.
Aviso: ID 99 inexistente.
Aviso: ID 25 duplicado (recusado).
Pacote 25 removido.
8 15 35
     35(P)
15(P)
     8(P)
Total de pacotes: 3 | Altura da arvore: 2
Total de rotacoes: 2 | Total de recoloracoes: 8
Invariantes preservadas (OK)
```

- Executa a mesma sequência da AVL, acrescentando o pacote 8.
- (V) indica nó vermelho e (P) nó preto.
- A entrada do 8 dispara recoloração de 15 e 35 para preto.
- Tratamento idêntico para duplicatas e buscas/remoções inexistentes.
- Validador confirma as 4 propriedades após todas as operações.

Testes realizados

7

Cenário de teste	Procedimento executado	Resultado
Árvore vazia	buscar(42), excluir(42), percurso em ordem, altura == 0	PASSOU (AVL e RB)
Único elemento	inserir(75), buscar(75), altura == 1, excluir(75)	PASSOU (AVL e RB)
Duplicados e inexistentes	inserir 20 duas vezes; buscar e excluir chave ausente 500	PASSOU (AVL e RB)
Remoção: 0, 1, 2 filhos e raiz	base {60,30,80,15,45,70,95,40,99} -> excluir 15, 45, 30, 60	PASSOU (AVL e RB)
Ordens crescente / decrescente / aleatória	1200 pacotes em cada modalidade de ordenação	PASSOU (AVL e RB)
Teste de estresse misto	15000 inserções/remoções aleatórias auditadas passo a passo	PASSOU (AVL e RB)

- Resultado consolidado da suíte: 50 testes executados, 0 falhas
- Auditoria automática: verifica ordem BST, alturas e fb em {-1,0,1} na AVL; raiz preta, ausência de conflito vermelho-vermelho e igualdade da altura negra em todos os ramos na Rubro-Negra
- Elementos restantes confrontados a cada operação contra um std::set de controle
- Ambas as estruturas recebem exatamente os mesmos vetores de entrada (semente fixa)

Análise comparativa: altura e rotações

Inserção de N pacotes seguida da remoção de 50% deles (mesmos conjuntos de dados nas duas árvores)

Volume / Ordem	AVL alt	RB alt	AVL rot ins	RB rot ins	RB recol ins	AVL rot rem	RB rot rem
1200 crescente	11	18	1189	1182	5907	98	110
1200 decrescente	11	18	1189	1182	5907	116	144
1200 aleatória	12	12	768	676	2718	227	234
80000 crescente	17	30	79983	79970	399850	6668	7373
80000 decrescente	17	30	79983	79970	399850	6765	7495
80000 aleatória	19	20	55983	46779	185539	16186	16265

Alturas exibidas após a inserção. Após remover metade: AVL 11/11/11 (N=1200) e 17/17/18 (N=80000); RB 11/11/12 e 18/18/19.

- Em entradas ordenadas (N=80000), a AVL mantém altura 17 contra 30 da Rubro-Negra (1,44 log n vs 2 log n)
- Em entradas aleatórias, as alturas ficam praticamente idênticas (19 na AVL vs 20 na Rubro-Negra)
- Na inserção aleatória (N=80000), a Rubro-Negra executa ~16,4% menos rotações (46779 vs 55983)

Análise comparativa: comportamento

- Árvore AVL: balanceamento estrito. Realiza mais rotações na inserção aleatória, mas garante menor altura no pior caso ordenado, reduzindo o número de comparações por busca
- Árvore Rubro-Negra: balanceamento mais permissivo. Substitui grande parte das rotações por simples trocas de cor (operações em bits, sem alterar ponteiros), acelerando inserções
- Na remoção de pacotes, ambas apresentaram quantidade similar de rotações (16186 na AVL contra 16265 na Rubro-Negra para 40000 exclusões aleatórias)

Benchmark: 500.000 inserções aleatórias + 500.000 buscas

Estrutura	Tempo (ms)	Altura final	Rotações	Recolorações
AVL	1020	22	349032	-
Rubro-Negra	918	23	291058	1157696

Tempos medidos com compilação otimizada (g++ -O2); valores absolutos variam conforme o hardware.

Recomendação técnica

9

Recomendação: Árvore AVL (Consultas) / Rubro-Negra (Escrita)

Para o núcleo de rastreamento da Amazon (leitura intensiva e lotes sequenciais), recomenda-se a AVL.

- Menor profundidade sob lotes ordenados: altura 17 contra 30 da Rubro-Negra (N=80000), reduzindo quase pela metade o caminho percorrido nas buscas quando IDs chegam sequencialmente
- No cenário puramente aleatório (500 mil pacotes), a Rubro-Negra foi ~10% mais rápida (918 ms vs 1020 ms) por realizar 57.974 rotações a menos e manter altura quase igual (23 vs 22)
- Custo de remoção equivalente entre as duas estruturas, não penalizando a escolha da AVL
- Conclusão para a engenharia da Amazon: adotar AVL nos serviços de consulta de status de encomendas e Rubro-Negra nas filas de ingestão/despacho com escrita massiva em tempo real

Referências e repositório

Repositório público no GitHub (código-fonte completo, executáveis, Makefile, README e testes):

https://github.com/rafaelcamara11/arvores-balanceadas

Referências

- CORMEN, T. H. et al. Algoritmos: Teoria e Prática. Elsevier (capítulo de Árvores Rubro-Negras)
- ADELSON-VELSKY, G. M.; LANDIS, E. M. An algorithm for the organization of information, 1962 (árvore AVL)
- GUIBAS, L.; SEDGEWICK, R. A dichromatic framework for balanced trees, 1978 (árvore Rubro-Negra)
- Notas de aula e materiais da disciplina IMD0039 - Estruturas de Dados Básicas II, UFRN/IMD

Autor: Rafael Medeiros Câmara da Silva - matrícula 20250067640